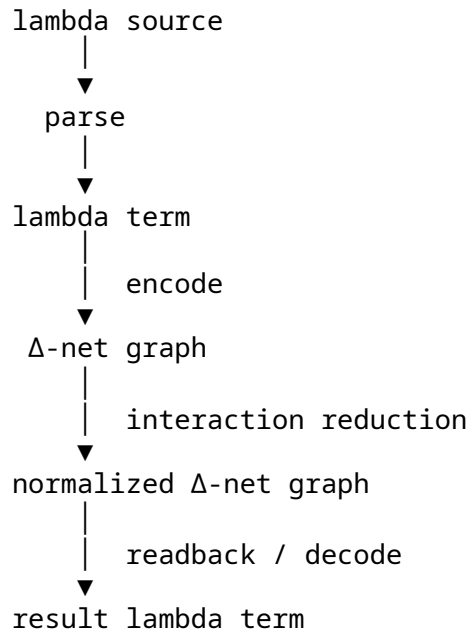


Next Steps: Encoding Lambda Terms as Δ -Nets

This guide uses **encoding** for the operation that constructs a Δ -net graph from lambda syntax.

1. Keep the four operations separate

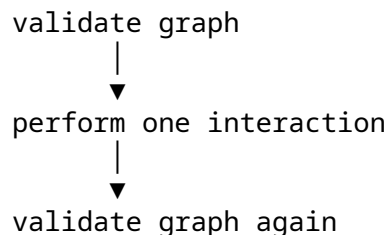


- **Interpretation:** the semantic meaning of a lambda term.
- **Encoding:** construction of agents, ports, wires, replicators, erasers, and interface ports from syntax.
- **Reduction:** interaction between agents.
- **Readback/decode:** reconstruction of lambda syntax from the graph.

Do not use interpretation for the graph-construction function. The concrete operation is encoding.

2. First milestone: make every rewrite trustworthy

Before encoding arbitrary terms, validate every existing interaction before and after it runs:



The validator must check:

- every live port has exactly one owner;
- every live port has exactly one wire;
- every wire has two distinct endpoints;
- every wire joins PARENT to CHILD;
- every active pair is principal-principal;
- every replicator auxiliary has polarity opposite its principal;

- auxiliary order and metadata order agree;
- no deleted port remains referenced;
- no auxiliary slice has two owners.

Fan-replicator interaction

The concrete port operations are:

```
fan.left  —reused as—> left_copy.principal
fan.right —reused as—> right_copy.principal

old_auxiliary[i] —cloned twice—> left_auxiliary[i]
                                ↳ right_auxiliary[i]

old_auxiliary[i] —reused as—> new_fan.principal
new_left  ————— left_auxiliary[i]
new_right ————— right_auxiliary[i]
```

The rules are:

reused port:
keep its polarity and change only its structural owner/slot as required

cloned port:
preserve source polarity and metadata, but receive a new identity/owner

new port:
receive the polarity opposite to the port at the other end of its wire

For a valid replicator:

```
old_auxiliary[i].polarity
== opposite(old_replicator.principal.polarity)

cloned_auxiliary[i].polarity
== old_auxiliary[i].polarity
== opposite(new_replicator.principal.polarity)
```

The clone does not need a fresh polarity calculation because copying preserves the already-valid replicator invariant.

3. Centralize graph mutation

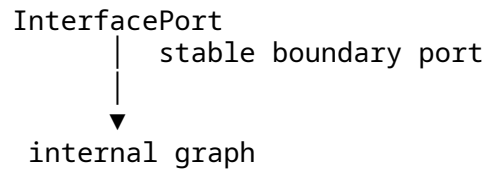
Reduce direct array manipulation in rewrite rules. Use a narrow mutation API:

```
new_fan()
new_eraser()
new_replicator(level, ordered_deltas, status, orientation)
new_interface_port(kind, name)
clone_port(source, owner)
connect(port_a, port_b)
disconnect(wire)
splice(port_a, port_b)
other_endpoint(wire, port)
destroy_node(node)
```

Each operation should update ownership, slot, polarity, wire IDs, and back-references together.

4. Preserve interface ports as stable boundaries

An InterfacePort is not an active agent and never reduces by itself:



The internal endpoint can change after reduction:

before: InterfacePort — fan

after: InterfacePort — new internal structure

The interface port itself keeps:

kind
name
port identity
port polarity

Use these conventions:

ROOT → PARENT
FREE_VARIABLE → CHILD

5. Implement the lambda term representation

Use a small De Bruijn representation:

```
Term :=
  BoundVar(index)
| FreeVar(name)
| Abstraction(body)
| Application(function, argument)
```

De Bruijn indices avoid relying on printable variable names during encoding:

$\lambda x.x$ → Abstraction(BoundVar(0))
 $\lambda x.\lambda y.x$ → Abstraction(Abstraction(BoundVar(1)))

Keep a simple normal-order reducer for reference results. It is the oracle for the graph machine, not part of the graph implementation.

6. Encode abstractions

An abstraction has one result and two semantic branches:

```
flowchart TB
    R[PARENT: abstraction result] --- P[PARENT: principal port]
    P --- F[Fan]
    F --- V[PARENT: variable x]
    F --- B[CHILD: body M]
```

Encoding steps:

1. Allocate a fan.
2. Connect its principal to the destination/result port.

3. Create the binder-side variable port.
4. Encode the body below the child/body port.
5. Track the binder while recursively encoding the body.

The fan is not intrinsically labelled “lambda.” Its port roles and rooted traversal determine the abstraction reading.

7. Encode applications

An application has one result and two child branches:

```

flowchart TB
    R[PARENT: application result] --- P[PARENT: principal port]
    P --- F[Fan]
    F --- FN[CHILD: function M]
    F --- A[CHILD: argument N]

```

Encoding steps:

1. Allocate a fan.
2. Connect its result/principal path to the destination port.
3. Encode the function under one child port.
4. Encode the argument under the other child port.

The application fan has two CHILD branches. The arrows in the diagram are wire layout only; polarity labels classify endpoints and do not describe runtime flow.

8. Encode variables

Bound variable

For a bound occurrence, find its binder in the encoding environment:

$\lambda x.x$

binder x ————— body occurrence x

If the variable is used once, a direct path is sufficient. If it is used more than once, introduce a replicator:

```

flowchart LR
    B[Binder port] --- R[Fan-in replicator]
    R --- U1[Use 1]
    R --- U2[Use 2]

```

Free variable

For an unbound name, create or reuse a named InterfacePort:

$\lambda x.y$

root — abstraction — InterfacePort(name = "y")

The name must survive encoding, reduction, and readback.

9. Encode erasure and sharing

For an unused binder:

$\lambda x.M$ where x does not occur in M

connect the unused binder port to an eraser rather than allocating a useless sharing structure:

flowchart LR

 B[Unused binder port] --- E[Eraser]

Use a replicator when one bound value must serve multiple occurrences or when level shifts require sharing metadata.

10. Implement readback before a full evaluator

Readback should work on hand-built graphs first:

root

- abstraction
- application
- bound variable
- free-variable InterfacePort
- replicator branch
- erased branch

The rooted traversal uses polarity and port roles:

enter principal/result path as PARENT
 → abstraction reading

enter application result path as PARENT
 → application reading

reach named InterfacePort
 → FreeVar(name)

Do not infer traversal order from wire-array allocation order.

11. Differential verification

For each sample:

source

- parse
- De Bruijn term
- reference normal-order reducer

source

- encode
- Δ -net reduction
- readback

compare both results up to alpha-equivalence

Start with:

$(\lambda x.x) y$	→ y
$(\lambda x.y) \Omega$	→ y
$(\lambda x.x x) (\lambda y.y)$	→ $\lambda y.y$
$\lambda x.x$	→ $\lambda x.x$

$\lambda x.\lambda y.x$	$\rightarrow \lambda x.\lambda y.x$
$\lambda x.x\ x$	$\rightarrow \lambda x.x\ x$

The next implementation milestone is therefore:

validated rewrites

- rooted traversal
- small De Bruijn term model
- abstraction/application encoding
- variable, eraser, and sharing encoding
- readback
- differential tests